

LabTrack

CAPSTONE PROJECT SUMMARY

SYSTEM TECHNOLOGY STACK

- **Frontend:** React 18 (Vite, single-page application)
- **Backend:** ASP.NET Core 8 Web API, C#
- **Database:** SQL Server, Entity Framework (EF) Core Migrations
- **Security:** JWT Bearer, Role-Based Access Control, BCrypt
- **AI & Data:** Pluggable RAG Pipeline, PdfPig (PDF extraction)
- **Testing:** xUnit, WebApplicationFactory, In-Memory DB

ARCHITECTURE & DESIGN PATTERNS

- **Layered Backend:** Implements a clean separation of concerns. Thin Controllers route HTTP requests to dedicated Services (Incident, Document, Rag, Admin, Auth), fully isolating business logic from external presentation.
- **Data Transfer Objects (DTOs):** DTOs are utilized across all endpoints to define exact JSON API contracts, preventing over-posting vulnerabilities and decoupling frontend structures from internal EF Core database models.
- **Pluggable Interface Design (DI):** AI dependencies (IEmbeddingService, ITextGenerationService) are injected via interfaces. This allows seamless swapping between a Local hashed-embedding provider and OpenAI via configuration, requiring zero core logic changes.
- **Global Error Handling:** Custom ExceptionHandlingMiddleware provides centralized, standardized HTTP error reporting.

CORE SYSTEM FEATURES

1. Incident Tracking Workflow

- **Validated State Machine:** Incidents are strictly governed by a legal lifecycle (Open → Investigating → Resolved → Closed). Invalid transitions are caught at the service layer, triggering 400 Bad Request errors.
- **Role-Aware Access:** Authorization is tiered across LabUser, Reviewer, and Administrator roles. Endpoints strictly enforce permissions for status changes, resolutions, and system administration.
- **Comprehensive Audit Trails:** Every status transition systematically writes to a relational StatusHistory table, logging the modifier, timestamps, and transitional states.
- **Database-Level Filtering:** Search and filter queries execute via EF Core LINQ clauses, delegating processing to SQL Server and returning standardized PagedResult sets.

2. Retrieval-Augmented Generation (RAG)

- **Semantic Retrieval:** Uploaded knowledge-base documents (.txt, .pdf) are segmented into overlapping text chunks (~120 words) and embedded as numeric vectors. User queries are compared via cosine similarity.
- **Strict Grounding Limits:** The generator synthesizes answers *exclusively* from retrieved contexts. If no relevant chunks surpass the similarity threshold, the AI explicitly refuses to answer, preventing dangerous safety hallucinations.
- **Source Traceability:** API responses include transparent sourcing (document title, chunk index, similarity score) and an isGrounded flag for UI warnings.

GOVERNANCE & DATA INTEGRITY

- **The Administrator Approval Gate:** A critical safety boundary. When a document is uploaded, it is chunked and stored as *Pending*. The RagService strictly filters for chunks where the parent DocumentItem.Status == Approved. The AI remains completely blind to unvetted material until explicitly authorized by an Administrator.
- **Relational Integrity:** The schema enforces Foreign Key relationships across Users, Incidents, Status Histories, Investigation Notes, Corrective Actions, Document Items, and Document Chunks.